

Code

```
#include <stdio.h>

#include <limits.h>


int main()
{
    int n = 3;

    int pid[] = {1, 2, 3};

    int at[] = {0, 2, 6};

    int bt[] = {10, 20, 30};

    int rt[3], ct[3], tat[3], wt[3];


    int complete = 0, t = 0, shortest;

    int minm = INT_MAX;

    int contextSwitch = 0;

    int prev = -1;


    // Copy burst time into remaining time
    for (int i = 0; i < n; i++)
        rt[i] = bt[i];


    printf("=====\n");
    printf("    SJF PREEMPTIVE SCHEDULING ALGORITHM\n");
    printf("=====\n\n");


    printf("INPUT PROCESSES:\n");
    printf("PID  Arrival  Burst\n");
    printf("-----\n");
    for (int i = 0; i < n; i++)
        printf("P%d   %d    %d\n", pid[i], at[i], bt[i]);
```

```

printf("\nEXECUTION SEQUENCE (Gantt Chart):\n");
printf("Time Process Remaining\n");
printf("-----\n");

while (complete != n)
{
    shortest = -1;
    minm = INT_MAX;

    for (int i = 0; i < n; i++)
    {
        if ((at[i] <= t) && (rt[i] < minm) && rt[i] > 0)
        {
            minm = rt[i];
            shortest = i;
        }
    }

    if (shortest == -1)
    {
        t++;
        continue;
    }

    if (prev != shortest && prev != -1)
        contextSwitch++;

    prev = shortest;

    printf("%d    P%d    %d\n", t, pid[shortest], rt[shortest]);

```

```

rt[shortest]--;
minm = rt[shortest];

if (rt[shortest] == 0)
{
    complete++;
    ct[shortest] = t + 1;
}

t++;
}

int total_wt = 0, total_tat = 0;

for (int i = 0; i < n; i++)
{
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\n=====\\n");
printf("RESULTS & CALCULATIONS\\n");
printf("=====\\n\\n");

printf("PROCESS EXECUTION DETAILS:\\n");
printf("PID  AT  BT  CT  TAT  WT\\n");
printf("-----\\n");

```

```

for (int i = 0; i < n; i++)

    printf("P%d  %d  %d  %d  %d  %d\n",

        pid[i], at[i], bt[i], ct[i], tat[i], wt[i]);


printf("\nPERFORMANCE METRICS:\n");
printf("-----\n");
printf("Total Context Switches: %d\n", contextSwitch);
printf("Total Waiting Time: %d\n", total_wt);
printf("Total Turnaround Time: %d\n", total_tat);
printf("Average Waiting Time: %.2f\n", (float)total_wt / n);
printf("Average Turnaround Time: %.2f\n", (float)total_tat / n);


printf("\n===== \n");
printf("VERIFICATION CALCULATIONS\n");
printf("===== \n\n");


for (int i = 0; i < n; i++)

{
    printf("P%d: CT=%d, TAT=%d, WT=%d\n",

        pid[i], ct[i], tat[i], wt[i]);
}


return 0;
}

```

```

=====
SJF PREEMPTIVE SCHEDULING ALGORITHM
=====

```

INPUT PROCESSES:

PID Arrival Burst

```

-----
P1    0        10
P2    2        20
P3    6        30

```

EXECUTION SEQUENCE (Gantt Chart):

Time Process Remaining

```

-----
0      P1        10
1      P1         9
2      P1         8
3      P1         7
4      P1         6
5      P1         5
6      P1         4
7      P1         3
8      P1         2
9      P1         1
10     P2        20
11     P2        19
12     P2        18
13     P2        17
14     P2        16
15     P2        15
16     P2        14
17     P2        13
18     P2        12
19     P2        11
20     P2        10
21     P2         9
22     P2         8
23     P2         7
24     P2         6
25     P2         5
26     P2         4
27     P2         3
28     P2         2

```

41	P3	19
42	P3	18
43	P3	17
44	P3	16
45	P3	15
46	P3	14
47	P3	13
48	P3	12
49	P3	11
50	P3	10
51	P3	9
52	P3	8
53	P3	7
54	P3	6
55	P3	5
56	P3	4
57	P3	3
58	P3	2
59	P3	1

=====

RESULTS & CALCULATIONS

=====

PROCESS EXECUTION DETAILS:

PID	AT	BT	CT	TAT	WT
-----	----	----	----	-----	----

P1	0	10	10	10	0
P2	2	20	30	28	8
P3	6	30	60	54	24

PERFORMANCE METRICS:

Total Context SwitcheS: 2
 Total Waiting Time: 32
 Total Turnaround Time: 92
 Average Waiting Time: 10.67
 Average Turnaround Time: 30.67

=====

VERIFICATION CALCULATIONS

=====

P1: CT=10, TAT=10, WT=0

